

Axyom-form 项目分析报告

一、项目概述

@axyom-ui/form 是一个基于 Angular 20 + ng-zorro-antd 20 封装的**配置驱动型动态表单框架**。通过声明式的 TypeScript 配置类（如 `StringUnit`、`SelectUnit`）描述表单字段，自动生成对应的 ng-zorro UI 表单，彻底告别手写重复模板代码的时代。

核心数据

指标	数值
Angular 版本	20.x (最新)
ng-zorro 版本	20.x
支持组件类型	21 种
自定义验证器	10 种
测试用例	46+
打包方式	ng-packagr (FESM)

二、技术架构

2.1 分层架构设计



2.2 核心设计模式

1. 注册表模式 (Registry Pattern)

```
// form-unit-registry.service.ts
@Injectable({ providedIn: 'root' })
export class FormUnitRegistryService {
  private units: Record<string, Type<any>> = {
    string: StringUnitComponent,
    number: NumberUnitComponent,
    select: SelectUnitComponent,
    // ... 21种组件
  };

  getFormUnit(type: string): Type<any> {
    return this.units[type] ?? this.units['string']; // 兜底策略
  }

  // 支持运行时扩展
  register(type: string, formUnit: Type<any>): void {
    this.units[type] = formUnit;
  }
}
```

设计亮点:

- 开闭原则: 新增组件无需修改现有代码
- 运行时可扩展: 支持第三方组件注册
- 优雅降级: 未知类型自动回退到 string 类型

2. 模板方法模式 (Template Method)

```
// FormBase 基类定义算法骨架
export abstract class FormBase<T> {
  // 子类必须实现
  abstract readonly controlType: string;

  // 模板方法
  protected getValid(instance): ValidatorFn[] {
    const valid = [];
    if (instance.valid) {
      Array.isArray(instance.valid)
        ? valid.push(...instance.valid)
        : valid.push(instance.valid);
    }
    if (instance.required) {
      valid.push(validators.required);
    }
    return valid;
  }
}
```

```

    }
  }

  // StringUnit 重写扩展验证
  export class StringUnit extends FormBase<string> {
    protected override getValid(instance): ValidatorFn[] {
      const valid = super.getValid(instance); // 调用父类
      if (instance.minLength) valid.push(Validators.minLength(instance.minLength));
      if (instance.maxLength) valid.push(Validators.maxLength(instance.maxLength));
      return valid;
    }
  }
}

```

3. 建造者模式 (Builder Pattern)

```

// dialog-modal.ts - 链式调用构建复杂对象
DialogModal.builder()
  .setTitle('确认删除')
  .setContent('确定要删除这条记录吗? ')
  .setOkText('确定')
  .setOkDanger(true)
  .setOk(() => this.deleteService.delete())
  .setCancelText('取消')
  .setWidth(400);

```

4. 策略模式 (Strategy Pattern)

```

// 条件显示策略 - 支持布尔值和函数两种策略
display: ((form: any) => boolean) | boolean = true;

// 静态策略
new StringUnit({ key: 'p1', display: false }) // 始终隐藏

// 动态策略
new StringUnit({
  key: 'p2',
  display: (formValue) => formValue.p1 !== 2
})

```

三、核心技术亮点

3.1 动态组件渲染引擎

技术选型演进：

```

ngIf + ngComponentOutlet (低效)
    ↓ 优化
NgComponentOutlet + computed (高效)

```

```
// form-unit.component.ts
@Component({
  template: `
    @if (fb().show) {
      <ng-container
        *ngComponentOutlet="component; inputs: formInput()" />
    }
  `
})
export class FormUnitComponent {
  readonly formInput = computed(() => ({
    fb: this.fb(),
    formGroup: this.form(),
  }));

  ngOnInit() {
    this.component = this.getFormItemComponent();
  }
}
```

优化点:

1. `computed` 缓存输入对象，避免每次变更检测重新创建
2. 仅在 `show=true` 时渲染，减少不必要的组件实例化
3. 使用 `NgComponentOutlet` 替代多个 `ngIf` 分支

3.2 类型安全的配置推断

```
// 类型定义
type BaseInf<T, R = { key: string }> = Omit<
  Partial<Omit<T, keyof R>> & R,
  'controlType' | 'control' | 'view'
>;

// 使用示例 - 完整的类型提示和自动补全
new StringUnit({
  key: 'username',           // ✓ 必填
  label: '用户名',          // ✓ 自动补全
  required: true,            // ✓ 布尔类型
  maxLength: 50,             // ✓ 数字类型
  placeholder: '请输入',    // ✓ 字符串类型
  // controlType: 'xxx',    // ✗ 编译时报错，禁止设置
});
```

3.3 条件显示与 FormControl 动态管理

```
// tool/form.ts
export function updateShow(form: FormGroup, fbs: FormBase[], value: any = null): void {
  if (value == null) {
    value = form.getRawValue();
  }
}
```

```

fb.forEach((fb) => {
  // 计算显示状态
  fb.show = typeof fb.display == 'boolean' ? fb.display : fb.display(value);

  if (fb.show) {
    // 动态添加 FormControl
    if (!form.controls[fb.key]) {
      form.addControl(fb.key, fb.control);
    }
  } else {
    // 动态移除 FormControl
    if (form.controls[fb.key]) {
      form.removeControl(fb.key);
    }
  }
});
}

```

关键设计:

- 隐藏字段的 FormControl 被移除，不会参与表单验证
- 显示时自动恢复 FormControl，保留原有值和验证状态
- 订阅 `valueChanges` 实现响应式更新

3.4 跨字段联动验证器

```

// valid/equal-to.ts
export const equalTo = (targetFb: FormBase): ValidatorFn => {
  return (control: AbstractControl): ValidationResult => {
    if (isEmptyInputValue(control.value)) {
      return null;
    }

    // 关键：订阅目标字段的变化，触发自身重新验证
    targetFb.control.valueChanges
      .pipe(takeUntilDestroyed())
      .subscribe(() => {
        control.updateValueAndValidity({ onlySelf: false });
      });

    return control.value === targetFb.control.value
      ? null
      : { equalTo: `The value must be equal to ${targetFb.label}` };
  };
};

```

技术难点突破:

- Angular 原生验证器不支持跨字段监听
- 通过订阅目标字段的 `valueChanges` 实现联动
- 使用 `takeUntilDestroyed()` 防止内存泄漏

3.5 超大数值范围验证 (突破 JS 精度限制)

```
// valid/big-range.ts
export const bigRange = (
  params: { min?: string; max?: string },
  isInt = false
): ValidatorFn => {
  return (control: AbstractControl): ValidationResult => {
    if (isEmptyInputValue(control.value)) return null;

    // 使用 BigInt 处理超大数值
    const bigValue = BigInt(control.value);

    if (params.min !== null && bigValue < BigInt(params.min)) {
      return { bigRange: `value must be >= ${params.min}` };
    }
    if (params.max !== null && bigValue > BigInt(params.max)) {
      return { bigRange: `value must be <= ${params.max}` };
    }
    return null;
  };
};
```

解决的问题:

- JavaScript `Number` 类型最大安全整数为 `253 - 1`
- 使用 `BigInt` 支持任意精度整数运算
- 适用于金融、加密等需要超大数值的场景

3.6 异步选项加载机制

```
// option-base.ts
export abstract class OptionBase<T> extends FormBase<T> {
  options: Option[] = [];
  load: (() => Observable<OptionInf>) | null = null;
  isLoading = false;

  override toView() {
    if (isEmptyInputValue(this.control.value)) {
      this.view.set('');
    }

    // 策略1: 本地选项直接转换
    if (this.options.length > 0) {
      this.view.set(this.getOptionLabels());
    }

    // 策略2: 异步加载
    else if (this.load !== null) {
      if (!this.isLoading) {
        this.isLoading = true;
        this.load()
      }
    }
  }
}
```

```
        .pipe(finally(() => this.isLoading = false))
        .subscribe((data) => {
            this.options = toOptions(data);
            this.view.set(this.getOptionLabels());
        });
    }
}
}
```

支持的加载模式：

- 1. **同步加载**：组件初始化时一次性加载
- 2. **异步加载**：支持 Observable 流式加载
- 3. **分页加载**：`SelectLoadUnit` 支持无限滚动分页

四、性能优化措施

4.1 编译时优化

优化项	实现方式	效果
Tree-shaking	<code>sideEffects: false</code>	移除未使用代码
独立组件	<code>standalone: true</code>	按需导入模块
懒加载	<code>loadComponent</code>	路由级代码分割

4.2 运行时优化

优化项	实现方式	效果
Signal 响应式	<code>signal()</code> / <code>computed()</code>	精准更新
列表优化	<code>@for track fb.key</code>	减少 DOM 操作
订阅管理	<code>takewhile</code> / <code>takeUntilDestroyed</code>	防止内存泄漏
计算缓存	<code>computed()</code>	避免重复计算

4.3 包体积优化

```
// package.json
{
  "sideEffects": false, // 支持 tree-shaking
  "ngPackage": {
    "lib": {
      "entryFile": "public-api.ts"
    }
  }
}
```

五、项目难点分析

5.1 动态组件的类型安全

难点： `NgComponentOutlet` 的 `inputs` 属性是 `Record<string, any>`，丢失了类型信息。

解决方案：

```
// 使用泛型约束基类
@Directive()
export class FormBaseUnit<T extends FormBase<R>, R> {
  readonly fb = input.required<T>();
  readonly formGroup = input.required<FormGroup>();
}

// 具体组件继承时获得类型提示
export class StringUnitComponent extends FormBaseUnit<StringUnit, string> {}
```

5.2 条件显示的状态同步

难点：

- 字段显示/隐藏需要动态添加/移除 `FormControl`
- 隐藏字段的值需要保留，显示时恢复
- 验证状态需要正确传递

解决方案：

- 使用 `form.getRawValue()` 获取所有值（包括禁用字段）
- 动态 `addControl` / `removeControl` 管理表单控件
- 订阅 `valueChanges` 实现响应式更新

5.3 跨字段验证的触发时机

难点：Angular 验证器是纯函数，无法主动触发其他字段重新验证。

解决方案：

```
// 在验证器内部订阅目标字段变化
targetFb.control.valueChanges
  .pipe(takeUntilDestroyed())
  .subscribe(() => {
    control.updateValueAndValidity({ onlySelf: false });
  });
```

5.4 异步回调的 Loading 状态管理

难点：按钮点击后需要自动管理 loading 状态，防止重复提交。

解决方案：

```
// dialog-modal.ts
executeFun(fun: any) {
  if (fun !== null) {
    const obs = fun();
    if (obs instanceof Observable) {
      return firstValueFrom(
        obs.pipe(catchError(() => of(true)))
      );
      // ng-zorro 会自动等待 Promise resolve 后关闭 loading
    }
  }
  return null;
}
```

六、面试技术亮点

6.1 架构设计能力

问题：请描述你的表单库架构设计

回答要点：

1. **五层架构：**基础设施层 → 基类层 → 组件层 → 调度层 → 容器层
2. **设计模式：**注册表模式、模板方法、建造者模式、策略模式
3. **扩展性：**支持自定义组件注册、自定义验证器、全局配置注入

6.2 Angular 高级特性

问题：你用了哪些 Angular 高级特性？

回答要点：

1. **Signal 响应式**：`signal()`、`computed()`、`effect()` 替代传统 RxJS
2. **动态组件**：`NgComponentOutlet` + 注册表实现运行时组件分发
3. **依赖注入**：`InjectionToken` + `provideAxyomFormConfig()` 实现全局配置
4. **新控制流**：`@if`、`@for`、`@switch` 替代旧指令
5. **Standalone 组件**：所有组件独立导入，按需加载

6.3 TypeScript 高级用法

问题：你如何保证配置的类型安全？

回答要点：

1. **泛型约束**：`FormBase<T>`、`FormBaseUnit<T, R>`
2. **工具类型**：`Omit`、`Partial` 组合创建配置类型
3. **抽象类**：`abstract controlType` 强制子类实现
4. **方法重写**：`override getValid()` 扩展验证逻辑

6.4 响应式编程

问题：你如何处理异步数据流？

回答要点：

1. **RxJS 操作符**：`finalize`、`catchError`、`takeUntilDestroyed`
2. **Promise 转换**：`firstValueFrom` 将 Observable 转为 Promise
3. **内存管理**：所有订阅都有明确的取消机制

6.5 性能优化

问题：你做了哪些性能优化？

回答要点：

1. **编译时**：Tree-shaking、Standalone 组件、懒加载
 2. **运行时**：Signal 精准更新、`computed` 缓存、`track` 优化列表
 3. **包体积**：`sideEffects: false`、按需导入
-

七、技术体系总结

7.1 技术栈全景



7.2 核心能力矩阵

能力维度	实现方式	技术深度
类型安全	泛型 + 抽象类 + 工具类型	★★★★
动态渲染	NgComponentOutlet + 注册表	★★★★
响应式	Signal + RxJS + computed	★★★★★
可扩展性	依赖注入 + 组件注册	★★★★
验证体系	自定义验证器 + 跨字段联动	★★★★
性能优化	Tree-shaking + 懒加载 + 缓存	★★★★

7.3 面试价值点

1. 架构思维：从需求分析到分层设计，体现系统性思考

2. 源码理解：深入 Angular 内部机制（变更检测、DI、动态组件）

3. 工程化：CI/CD、单元测试、包管理、版本发布

4. 问题解决：跨字段验证、超大数值、异步加载等实际问题

5. 最佳实践：响应式编程、内存管理、性能优化

八、面试深度亮点（技术深度展示）

8.1 设计模式实战应用

问题：你用了哪些设计模式？为什么这样设计？

注册表模式 - 实现开闭原则

// 问题：新增表单类型需要修改多处代码
// 解决：集中管理类型→组件的映射关系

// 1. 定义注册表

```

@Injectables({ providedIn: 'root' })
export class FormUnitRegistryService {
  private units = new Map<string, Type<any>>();

  // 2. 支持运行时扩展（微前端场景）
  register(type: string, component: Type<any>): void {
    if (this.units.has(type)) {
      console.warn(`Type "${type}" already registered, will be overwritten`);
    }
    this.units.set(type, component);
  }

  // 3. 优雅降级策略
  getFormUnit(type: string): Type<any> {
    return this.units.get(type) ?? this.units.get('string')!;
  }
}

// 4. 使用方无需关心实现细节
@Component({ template: `<ng-container *ngComponentOutlet="component" />` })
export class FormUnitComponent {
  component = this.registry.getFormUnit(this.fb().controlType);
}

```

面试追问点：

- Q: 为什么不用 `switch-case` 或 `ngSwitch` ?
- A: 注册表模式支持运行时扩展，`switch` 是编译时静态绑定，无法支持微前端或插件化场景

模板方法模式 - 实现代码复用

```

// 问题：每个组件都要写验证逻辑，代码重复
// 解决：基类定义算法骨架，子类重写扩展

export abstract class FormBase<T> {
  // 1. 基类实现通用验证逻辑
  protected getValid(instance: BaseInf<FormBase<T>>): ValidatorFn[] {
    const valid: ValidatorFn[] = [];
    if (instance.valid) {
      Array.isArray(instance.valid)
        ? valid.push(...instance.valid)
        : valid.push(instance.valid);
    }
    if (instance.required) valid.push(validators.required);
    return valid;
  }
}

// 2. 子类重写扩展特有验证
export class StringUnit extends FormBase<string> {
  protected override getValid(instance: BaseInf<StringUnit>): ValidatorFn[] {

```

```

    const valid = super.getValid(instance); // 调用父类
    if (instance.minLength) valid.push(Validators.minLength(instance.minLength));
    if (instance.maxLength) valid.push(Validators.maxLength(instance.maxLength));
    return valid;
  }
}

export class NumberUnit extends FormBase<number> {
  protected override getValid(instance: BaseInf<NumberUnit>): ValidatorFn[] {
    const valid = super.getValid(instance);
    if (instance.min !== null) valid.push(Validators.min(instance.min));
    if (instance.max !== null) valid.push(Validators.max(instance.max));
    return valid;
  }
}

```

设计原则体现：

- **开闭原则**：新增组件类型只需继承 FormBase，无需修改现有代码
- **里氏替换**：所有子类都可以替换父类使用
- **单一职责**：基类只管通用验证，子类只管特有验证

8.2 Angular 源码级理解

问题：Signal 和 RxJS 的区别？为什么混用？

```

// 1. Signal - 状态管理（同步、细粒度）
readonly view = signal(''); // 组件状态

// 2. RXJS - 异步流处理
this.form()
  .valueChanges
  .pipe(takewhile(() => this.alive))
  .subscribe(() => updateShow(this.form(), this.fbs()));

// 3. 混用场景：Signal 驱动的 computed
readonly formInput = computed(() => ({
  fb: this.fb(), // Signal 依赖追踪
  formGroup: this.form(),
}));

// 4. effect - 副作用处理
effect(() => {
  const options = this.fb().options(); // 自动追踪依赖
  // 当 options 变化时执行
});

```

面试回答框架：

维度	Signal	RxJS
本质	响应式状态容器	异步数据流
粒度	细粒度更新	粗粒度订阅
同步/异步	同步读取	异步流
适用场景	组件状态、模板绑定	HTTP请求、事件流、定时器
内存管理	自动	需手动取消订阅

结论： 我在库中用 Signal 管理组件状态（view、formInput），用 RxJS 处理异步操作（valueChanges、load），各取所长。

问题： NgComponentOutlet 和 ngSwitch 有什么区别？ 为什么选前者？

```
// 方案1: ngSwitch (编译时确定, 运行时切换)
@switch (type) {
  @case ('string') { <string-unit /> }
  @case ('number') { <number-unit /> }
  @case ('select') { <select-unit /> }
  // ... 21个 case, 代码膨胀
}

// 方案2: NgComponentOutlet (运行时动态决定)
<ng-container
  *ngComponentOutlet="component; inputs: formInput()" />

// 组件通过注册表动态获取
component = this.registry.getFormUnit(this.fb().controlType);
```

技术决策理由：

- 1. **可扩展性：** ngSwitch 需要编译时知道所有类型, NgComponentOutlet 支持运行时注册
- 2. **代码量：** 21 种组件用 ngSwitch 需要 21 个 case, NgComponentOutlet 只需 1 行
- 3. **性能：** NgComponentOutlet 只实例化需要的组件, ngSwitch 虽然也是按需渲染但模板更复杂
- 4. **微前端：** NgComponentOutlet 支持动态加载远程组件

问题： 依赖注入在库中怎么用的？

```
// 1. 全局配置注入
export const AXYOM_FORM_CONFIG = new InjectionToken<AxyomFormConfig>('AXYOM_FORM_CONFIG');
```

```
// 2. 提供者函数
export function provideAxyomFormConfig(config: AxyomFormConfig) {
  return { provide: AXYOM_FORM_CONFIG, useValue: config };
}
```

```
// 3. 应用层配置
// app.config.ts
providers: [
  provideAxyomFormConfig({
    dialog: { okText: '确认', cancelText: '取消' },
    formModal: { width: 600 }
  })
]

// 4. 库内部消费
@Injectable({ providedIn: 'root' })
export class DynamicModalService {
  private config = inject(AXYOM_FORM_CONFIG); // 类型安全
}
```

面试加分点:

- InjectionToken 解决字符串 token 的类型安全问题
- providedIn: 'root' 实现单例, 无需额外配置
- providexxx 函数符合 Angular 最佳实践 (与 provideRouter 风格一致)

8.3 TypeScript 高级类型体操

问题: BaseInf 类型是怎么设计的? 为什么这么复杂?

```
// 目标: 配置对象只需要传 key, 其他属性可选
type BaseInf<T, R = { key: string }> = Omit<
  Partial<Omit<T, keyof R>> & R,
  'controlType' | 'control' | 'view'
>;
```

类型推导过程:

```
// 1. 假设 T = StringUnit
interface StringUnit {
  controlType: string; // 排除
  control: FormControl; // 排除
  view: Signal; // 排除
  key: string; // 必填
  label: string; // 可选
  required: boolean; // 可选
  maxLength: number; // 可选
}

// 2. Omit<T, keyof R> - 移除 key 属性
Omit<StringUnit, 'key'> // { controlType, control, view, label, required, maxLength }

// 3. Partial<...> - 所有属性变可选
Partial<Omit<StringUnit, 'key'>> // { controlType?, control?, view?, label?, required?,
maxLength? }
```

```
// 4. & R - 交叉类型, key 必填
Partial<Omit<StringUnit, 'key'>> & { key: string }

// 5. Omit<..., 'controlType' | 'control' | 'view'> - 移除内部属性
// 最终结果: { key: string; label?: string; required?: boolean; maxLength?: number; ... }
```

面试回答:

这个类型设计实现了"必填 key + 可选配置"的模式, 同时屏蔽了库内部使用的 `controlType`、`control`、`view` 属性, 防止用户误操作。

8.4 响应式编程深度

问题: 跨字段验证是怎么实现的? 内存泄漏怎么处理?

```
// 问题: 密码确认需要监听密码字段变化
export const equalTo = (targetFb: FormBase): ValidatorFn => {
  // 1. 闭包持有目标字段引用
  return (control: AbstractControl): ValidationResult => {
    if (isEmptyInputValue(control.value)) return null;

    // 2. 订阅目标字段的 valueChanges
    targetFb.control.valueChanges
      .pipe(takeUntilDestroyed()) // 3. 自动取消订阅
      .subscribe(() => {
        // 4. 目标字段变化时, 触发自身重新验证
        control.updateValueAndValidity({ onlySelf: false });
      });

    return control.value === targetFb.control.value
      ? null
      : { equalTo: `${targetFb.label} 不匹配` };
  };
};
```

内存泄漏防护:

```
// 方案1: takeUntilDestroyed (推荐, Angular 16+)
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
this.destroyRef = inject(DestroyRef);

targetFb.control.valueChanges
  .pipe(takeUntilDestroyed(this.destroyRef))
  .subscribe();

// 方案2: takeWhile (组件内使用)
private alive = true;
ngOnDestroy() { this.alive = false; }

this.form().valueChanges
  .pipe(takeWhile(() => this.alive))
```



```
.subscribe();
```

```
// 方案3: async pipe (模板中使用, 自动取消)  
{{ form.valueChanges | async }}
```

面试追问:

- Q: `takeUntilDestroyed` 和 `takeUntil` 的区别?
- A: `takeUntilDestroyed` 自动监听 `DestroyRef`, 无需手动创建 Subject; `takeUntil` 需要手动 `next()` + `complete()`

8.5 性能优化深度

问题: computed 缓存是怎么工作的?

```
// FormUnitComponent 中  
readonly formInput = computed(() => ({  
  fb: this.fb(),  
  formGroup: this.form(),  
}));  
  
// 模板中  
<ng-container *ngComponentOutlet="component; inputs: formInput()" />
```

Angular Signal 变更检测流程:

1. `fb()` 或 `form()` 变化
↓
2. Angular 标记 `computed` 为 `dirty`
↓
3. 下次访问 `formInput()` 时重新计算
↓
4. 如果结果与上次相同, 不触发组件更新
↓
5. `NgComponentOutlet` 接收到新 `inputs`, 按需更新子组件

对比传统方式:

```
// 传统方式: 每次变更检测都创建新对象  
get formInput() {  
  return { fb: this.fb, formGroup: this.form }; // 每次都 new  
}  
  
// signal 方式: 只有依赖变化时才重新计算  
readonly formInput = computed(() => ({...})); // 缓存结果
```

性能收益:

- 减少对象创建次数

- 减少 NgComponentOutlet 的 input 变更检测
- 子组件只在 inputs 真正变化时更新

问题：为什么用 sideEffects: false?

```
// package.json
{
  "sideEffects": false
}
```

Tree-shaking 原理:

```
// 1. 用户只使用了 StringUnit
import { StringUnit, toForm } from '@axyom-ui/form';

// 2. 有 sideEffects: false 时，打包工具会：
//    - 保留 StringUnit 和 toForm
//    - 移除其他未使用的组件（SelectUnit、NumberUnit 等）

// 3. 最终 bundle 只包含用户实际使用的代码
```

对比效果:

- 无 sideEffects: 整个库被打包 (约 50KB+)
- 有 sideEffects: 只打包使用的组件 (约 10KB)

8.6 工程化能力展示

问题：CI/CD 是怎么设计的?

```
# .gitlab-ci.yml
stages:
  - test
  - deploy

test:
  stage: test
  script:
    - pnpm install
    - pnpm run build:form
    - pnpm run test:form -- --code-coverage
  coverage: '/Lines\s*:\s*(\d+\.\?\d*)%/'

deploy:
  stage: deploy
  only:
    - main
  when: on-success # 测试通过才发布
  script:
```

```
- cd projects/form
- npm publish --registry
https://gitlab.com/api/v4/projects/${CI_PROJECT_ID}/packages/npm
```

工程化亮点:

1. **测试先行**: test 阶段必须通过才能 deploy
2. **覆盖率报告**: 自动提取覆盖率数据
3. **条件发布**: 仅 main 分支 + 测试通过时发布
4. **版本管理**: package.json 版本号控制发布

问题: 单元测试怎么写的?

```
// valid/ip.spec.ts
describe('ip validator', () => {
  it('should validate IPv4 address', () => {
    // 1. 创建 FormControl
    const control = new FormControl('192.168.1.1');

    // 2. 应用验证器
    const validator = ip('ipv4');
    const result = validator(control);

    // 3. 断言结果
    expect(result).toBeNull(); // 验证通过
  });

  it('should reject invalid IPv4', () => {
    const control = new FormControl('256.1.1.1');
    const validator = ip('ipv4');
    const result = validator(control);

    expect(result).toEqual({ ip: 'The input value should be IPv4' });
  });
});
```

测试策略:

- **纯逻辑测试**: 验证器是纯函数, 无需 TestBed
 - **边界覆盖**: 测试正常值、边界值、异常值
 - **快速反馈**: 单个测试 < 10ms
-

8.7 问题解决能力展示

难题1：条件显示的 FormControl 管理

问题描述：

字段 A 显示时，需要验证
字段 A 隐藏时，应该移除验证
字段 A 再次显示时，之前的值应该恢复

解决方案：

```
export function updateShow(form: FormGroup, fbs: FormBase[], value: any = null): void {
  if (value == null) value = form.getRawValue(); // 获取所有值（包括 disabled）

  fbs.forEach((fb) => {
    const shouldShow = typeof fb.display === 'boolean'
      ? fb.display
      : fb.display(value);

    if (shouldShow && !fb.show) {
      // 显示：添加 FormControl
      form.addControl(fb.key, fb.control);
      // control 保留了之前的值和验证状态
    } else if (!shouldShow && fb.show) {
      // 隐藏：移除 FormControl
      form.removeControl(fb.key);
      // control 对象仍然存在，只是从 FormGroup 移除
    }

    fb.show = shouldShow;
  });
}
```

关键点：

- `form.getRawValue()` 获取所有值，包括 disabled 字段
- `form.removeControl()` 只是从 FormGroup 移除，不销毁 control 对象
- 再次 `addControl` 时，control 的值和验证器仍然存在

难题2：异步回调的 Loading 状态自动管理

问题描述：

用户点击"确定"按钮

- 如果回调是同步函数，直接关闭弹窗
- 如果回调是 `Observable`，显示 `loading`，等待完成后关闭
- 如果 `Observable` 出错，隐藏 `loading`，不关闭弹窗

解决方案：

```

executeFun(fun: any) {
  if (fun !== null) {
    const result = fun(); // 执行回调

    // 检测是否返回 Observable
    if (result instanceof Observable) {
      // 转为 Promise, ng-zorro 会自动等待 Promise resolve
      return firstValueFrom(
        result.pipe(
          catchError(() => of(true)) // 错误时不关闭弹窗
        )
      );
      // Promise resolve → 关闭弹窗 + 隐藏 loading
      // Promise reject → 只隐藏 loading, 不关闭弹窗
    }
  }
  return null;
}

```

技术细节:

- ng-zorro 的 `nzOnOk` 支持返回 Promise, 会自动管理 loading
- `firstValueFrom` 将 Observable 转为 Promise
- `catchError(() => of(true))` 吞掉错误, 返回成功 (不关闭弹窗)

8.8 面试话术模板

开场白 (30秒)

"我做了一个基于 ng-zorro 封装的配置驱动型动态表单框架, 核心解决的问题是: **用 TypeScript 配置替代手写重复的模板代码**。通过声明式的配置类描述表单字段, 自动生成对应的 UI 表单。目前支持 21 种组件类型、10 种自定义验证器, 支持条件显示、异步选项加载、跨字段验证等高级功能。"

技术深度展示 (选择 2-3 个点深入)

点1: 架构设计

"我采用了五层架构设计, 底层是 FormBase 抽象类定义数据模型, 中间层是注册表服务实现动态组件分发, 顶层是容器组件管理布局。这种分层使得新增组件类型只需: 1) 继承 FormBase 定义配置类; 2) 继承 FormBaseUnit 实现 UI 组件; 3) 在注册表中注册。完全符合开闭原则。"

点2: 类型安全

"通过 TypeScript 泛型和工具类型, 实现了配置对象的完整类型推导。比如 `BaseInf<T>` 类型, 它使用 `Omit` 和 `Partial` 组合, 让配置对象只需要传 `key` 属性, 其他属性可选, 同时屏蔽了库内部使用的 `controlType`、`control` 等属性。"

点3: 性能优化

"使用 Angular Signal 的 `computed()` 缓存动态组件的 inputs 对象，只有当 `fb` 或 `form` 真正变化时才重新计算，避免了每次变更检测都创建新对象。配合 `sideEffects: false` 声明，实现了 tree-shaking，用户只引入需要的组件。"

收尾（15秒）

"这个项目让我深入理解了 Angular 的动态组件机制、响应式编程、依赖注入等核心特性，也锻炼了从需求分析到架构设计的系统性思维。"

8.9 常见面试问题清单

问题类型	问题	回答要点
架构设计	为什么选择配置驱动而不是模板驱动？	减少重复代码、类型安全、易维护
架构设计	如何支持自定义组件？	注册表模式 + Dependency Injection
Angular	Signal 和 RxJS 的区别？	状态 vs 流、同步 vs 异步、细粒度 vs 粗粒度
Angular	NgComponentOutlet 的原理？	运行时动态创建组件实例
Angular	变更检测机制？	Zone.js + DefaultStrategy / OnPush
TypeScript	泛型约束怎么用？	<code>T extends FormBase<R></code> 约束泛型范围
TypeScript	工具类型有哪些？	Partial、Required、Omit、Pick、Record
RxJS	takeUntilDestroyed 怎么用？	自动监听 DestroyRef，无需手动取消
性能	Tree-shaking 原理？	静态分析 import/export，移除未使用代码
性能	computed 缓存原理？	依赖追踪 + 惰性计算 + 结果缓存
测试	验证器怎么测试？	纯函数测试，无需 TestBed
工程化	CI/CD 流程？	test → build → deploy，条件发布

九、改进建议

9.1 短期优化

- 1. 补充文档：API 文档、使用指南、CHANGELOG
- 2. 增强测试：组件级别测试、E2E 测试
- 3. 错误处理：统一错误边界、用户友好的错误提示

9.2 中期演进

1. **Signal Forms**: 迁移到 Angular 21+ 的新 Signal Forms API
2. **Schema 驱动**: 支持 JSON Schema 自动生成表单
3. **国际化**: 支持多语言错误消息

9.3 长期规划

1. **可视化编辑**: 表单设计器, 拖拽式表单构建
2. **微前端支持**: 跨应用组件共享
3. **AI 辅助**: 智能表单生成、自动验证规则推荐